Semester Assignment – Reading Tracker Web Application

Last update: Oct 16, 2026

1. Overview

You will build a full-stack web application for a reader who currently keeps track, in a Word document, of the books they have read and the books they want to read. The application replaces that document with structured data, consistent input, and views that are computed automatically.

The current way of tracking is described in **`How_Books_Read_Currently_Tracked.pdf`** (provided with this statement). Read it carefully: it is your requirements source. It shows the real problems your application must solve:

- **Inconsistent structure.** Recent years record year, form, mode, content and theme for each book; older years only title, author and grade, even where the columns exist.
- **Mixed languages and inconsistent values.** "Romance" next to "Novel", "Prosa" next to "Prose", grades written as "7", "7,0", "7.5" and "7,5".
- **The same author spelled in different ways.** For example, Dostoevsky / Dostoiévski and Chekhov / Tchekhov.
- **Typos in titles and names.** For example, "Ray Bradburry" and "2001 - A Space Oddity".
- **Views maintained by hand that fall out of date.** The list of books by century and the lists of authors by nationality are incomplete and contain numbering errors. Some entries are also classified inconsistently, for example an author listed under a country different from the one used elsewhere.
- **A wishlist mixed with the reading log.** Books to read in the future sit at the end of the list, often without author, year or grade, sometimes with a note on who recommended them.
- Many entries lack a grade, a publication year or a classification, and nothing shows which ones still need attention.

The application's interface and its controlled values (forms, modes, statuses, priorities) are in US English. Book titles and author names are kept as recorded, in their original language. Internationalization is out of scope, but keep interface text in one place rather than scattered through the code, so it could be translated later.

2. Logistics

The assignment is worth **10 points** and has a **weight of 0.3** in the final course grade. It is split into two deliveries of 5 points each.

Delivery	Due date	Scope	Points
1. Frontend	November 20, 2026, 23:59 (Brasília time)	Complete user interface in **vanilla** HTML, CSS and JavaScript, with data stored in the browser	5.0
2. Full stack	December 11, 2026, 23:59 (Brasília time)	Spring Boot REST API, persistence, and the Delivery 1 frontend integrated with the API	5.0

- **GitHub**: Both deliverables must be made available in a publicly accessible GitHub repository.

- **Teams:** individual, or teams of at most **5 students**. The requirements are the same for every team size.
- **Team composition must be stated in the README of Delivery 1 and cannot change for Delivery 2.**
- **Seed data:** the instructor provides the reading log from the PDF as a JSON file (`seed.json`, see Section 11). Both deliveries must load it on first use: Delivery 1 into localStorage, Delivery 2 into the database.
- **Late submissions:** I grade the tagged commit (or, without a tag, the last commit pushed) and the video link in your submission email. A delivery is late if the email or the graded commit arrives after the deadline. **Submissions made up to 24 hours after the deadline** will incur a **20% penalty** on the assignment's total points. Commits and emails sent more than 24 hours late are ignored, so a delivery with nothing submitted within those 24 hours receives zero.

3. Technical constraints

The technology stack is fixed; only the persistence layer is your choice.

Layer	Required	Allowed	Not allowed
Frontend	Vanilla HTML, CSS and JavaScript; `fetch` for HTTP calls	`Chart.js` or a similar small charting library, loaded from a CDN or a local file; `Google Fonts`; `icon fonts`	Frameworks and UI libraries (React, Vue, Angular, Svelte, jQuery, Bootstrap, Tailwind, etc.); TypeScript; build tools and bundlers (npm packages, Vite, Webpack)
Backend	Java 21 or newer, Spring Boot 4.x, Maven or Gradle, REST API with JSON	Spring Web, Spring Data JPA, Bean Validation, Lombok; the database driver for your persistence choice (e.g., H2, including its console); Spring Boot DevTools; Spring Boot's test starter	Server-side HTML rendering (Thymeleaf, JSP)
Persistence	Data must survive an application restart	Any option; see Section 7 for the recommended one	Data kept only in memory

Any other library must be approved by the instructor before you use it; ask through *email*, and record the approval in your `README`.

4. Domain model

The application manages three main kinds of records: **authors**, **books** and **readings**. A book is the work itself; a reading is one time the person read (or plans to read) it. Keeping them separate allows a book to be read more than once and lets the wishlist and the reading log live in the same place.

Author

Field	Type	Required	Rules
Name	Text	Yes	Unique, ignoring upper/lower case
Nationality	Country, chosen from a fixed list	No	One country per author. The list is the one in the seed data; it cannot be changed through the application.
Birth date	Date	No	Not in the future

Field	Type	Required	Rules
Death date	Date	No	After the birth date

Book

Field	Type	Required	Rules
Title	Text	Yes	
Authors	Zero or more authors	No	Selected from existing authors, never retyped. A book may be saved with only its title and completed later.
Original publication year	Integer	No	Not in the future; negative values mean BC (e.g., Oedipus Rex, c. 429 BC). Year 0 is invalid, since there is no year 0 between 1 BC and AD 1.
Form	Chosen from a list	No	Novel, Novella, Short Story, Short Story Collection, Play, Essay, Treatise, Diary, Graphic Novel
Mode of expression	Chosen from a list	No	Prose, Drama, Poetry, Sequential Art
Content	Chosen from a list	No	Fiction, Non-fiction
Themes	Zero or more tags	No	Chosen from existing tags or created on the spot; no duplicates

Reading

Field	Type	Required	Rules
Book	Reference to a book	Yes	A book can have several readings
Status	Chosen from a list	Yes	Want to read, Reading, Paused, Read, Abandoned
Year read	Integer	Yes when status is Read; Optional when Abandoned; Empty otherwise	Not in the future; not earlier than the book's publication year
Grade	Decimal, 0.0 to 10.0	Only allowed when status is Read or Abandoned	Steps of 0.5
Recommended by	Text	No	Free text, e.g., a friend's name
Priority	Low, Medium, High	No	Only allowed when status is Want to read
Notes	Long text	No	

Themes and countries are simple records with an ID and a name. Theme names are unique, ignoring upper/lower case; a new theme can be created from the book form. Countries come from the seed data and cannot be created, edited or deleted through the application.

The lists above (forms, modes, content, statuses, priorities) are the minimum. You may add values, but each value must be stored and displayed in one consistent spelling.

5. Functional requirements

Core requirements (FR-01 to FR-07) are mandatory in both deliveries. Section 13 lists optional exercises. In your `README` file at GitHub, list each requirement ID and whether you implemented it.

Core Requirements

- **FR-01 Manage authors.** Create, view, edit and delete authors. An author linked to a book cannot be deleted; instead, show a message that names the books linked to that author.
- **FR-02 Manage books.** Create, view, edit and delete books. Authors and themes are selected from existing records (with the option to create a new theme on the spot).
- **FR-03 Manage readings.** Add one or more readings to a book; edit and delete them. The book form also has an optional section for the book's first reading (status, priority, recommended by, and so on). Saving the form creates both the book and that reading, so the user can add a book to the wishlist, or log a book already read, with a single Save (A book on the wishlist is really a book plus one reading with status Want to read.). Deleting a book also deletes its readings, after the user confirms; the confirmation says how many readings will be deleted
- **FR-04 Status workflow.** A reading can move between **Want to read**, **Reading**, **Paused**, **Read** and **Abandoned**. Paused means the book was started and will be resumed later. Moving to Read requires a year read. When a reading leaves Want to read, its recommendation and notes are kept; its priority is cleared. If a reading moves to a status that does not allow a year read or a grade, those values are cleared after the user confirms. To record a second reading of the same book, add a new reading instead of changing the status of the old one.
- **FR-05 Reading log.** A list of readings showing title, authors, status, year read and grade, sortable by each of these columns. Missing values are shown as "-", and sort after all filled values. By default, the log is sorted by year read (most recent first), then by ID, ascending (the order in which the readings were entered).
- **FR-06 Validation.** Every rule in Section 4 is enforced in the form, with a message next to the field that failed. Invalid data is never saved.
- **FR-07 Seed data.** On first use, the application loads the instructor-provided seed data. Later visits keep the user's changes and never load the seed again. A Reset data action, after confirmation, deletes all data and loads the seed again.

6. Delivery 1 – Frontend (November 20)

Deliver the complete user interface, implementing **FR-01 to FR-07**, with no backend. Data lives in the browser.

- **Data storage.** Place `seed.json` in your project (for example, `data/seed.json`). On first load, read it with `fetch` and save its contents in `localStorage`; on later loads, read only from `localStorage`. All changes are saved in `localStorage`, so they survive a page reload. New records get the next integer ID after the highest existing one. Reset data (FR-07) clears your `localStorage` keys and loads `seed.json` again.
- **Serving the pages.** `fetch` does not work when pages are opened directly from disk (`file://`). Serve the folder with a simple static server, such as the **VS Code *Live Server* extension**. Keep all frontend files in one folder; in Delivery 2 that folder is copied into `src/main/resources/static`

- **Prepare for Delivery 2.** Put all data access in a single JavaScript module (for example, `store.js`) with functions like `getBooks()`, `saveBook(book)` and `deleteBook(id)`. Make every store function `async` (it returns a `Promise`) even though `localStorage` does not need it, and call it with `await`. Use the data shapes of `seed.json`, which are also the shapes your API will return. When saving fails, throw an error with the same structure your API will send in a 400 response (each invalid field and the reason). **In Delivery 2 you will rewrite only this module to call the API; the rest of the code should not change.**
- **Code organization.** Separate files for HTML pages, CSS and JavaScript, with JavaScript split into files by responsibility: data access, rendering and validation (and charts, if you implement FR-13). No inline `onclick` or `style` attributes and no `<style>` blocks inside HTML; attach events with `addEventListener`. You may connect your JavaScript files in either of two ways; both are graded equally:
    - **Plain scripts**. Load each file with its own `<script src="…" defer>` tag, in dependency order (`store.js` before the files that use it). All files share one global scope, so give functions distinct names.
    - **ES modules (ECMAScript modules - recommended)**. Each file shares functions with `export` and uses other files' functions with `import`, and the page loads one entry file with `<script type="module" src="…">`. The browser then finds the other files itself, so there's no load order to manage and no name clashes. See MDN's JavaScript modules guide, sections "**Exporting module features**", "**Importing features into your script**" and "**Applying the module to your HTML**". **Note:** The MDN guide is long, these are the three sections students actually need. They cover everything required for this assignment.

7. Delivery 2 – Full stack (December 11)

Add a Spring Boot backend, store the data in a persistence layer, and switch the Delivery 1 frontend from `localStorage` to the `REST API`. Every Core feature from Delivery 1 (FR-01 to FR-07) must keep working.

REST API

The minimum set of **endpoints** is below. You may add more, and you may rename paths, as long as they follow REST conventions and are listed in your `README`. Request and response bodies use the same JSON shapes as `seed.json` (for example, a book refers to its authors and themes by `authorIds` and `themeIds`).

Method	Path	Purpose
GET, POST	/api/authors	List authors; create an author
GET, PUT, DELETE	/api/authors/{id}	Read, update, delete one author
GET, POST	/api/books	List books; create a book
GET, PUT, DELETE	/api/books/{id}	Read, update, delete one book (deleting also deletes its readings)
GET, POST	/api/books/{id}/readings	List a book's readings; add a reading
GET	/api/readings	List all readings, for the reading log

PUT, DELETE	/api/readings/{id}	Update, delete one reading
GET, POST	/api/themes	List themes; create a theme
GET	/api/countries	List countries (fixed list, for the nationality field)
POST	/api/reset	Delete all data and load the seed again (FR-07)

If you implement the optional views (FR-10 to FR-13), they may be computed in the backend (extra endpoints) or in the frontend from API data.

- **Status codes.** Use 200, 201, 204, 400, 404, and 409 appropriately (e.g., 409 when deleting an author linked to a book, or when creating an author or theme whose name already exists).
- **Validation.** All Section 4 rules are checked again on the server. A rejected request returns 400 with a body in the format below, and the frontend shows each message next to its field. 404 and 409 responses use the same format, with their own status and message and an empty errors list.

```
{
  "status": 400,
  "message": "Validation failed",
  "errors": [
    { "field": "yearRead", "message": "Year read is required when status is Read" },
    { "field": "grade", "message": "Grade must be between 0.0 and 10.0 in steps of 0.5" }
  ]
}
```

For example, deleting an author linked to a book returns:

```
{
   "status": 409,
   "message": "Author is linked to 2 books: Crime and Punishment, The Idiot",
   "errors": []
}
```

- **Layers.** Organize the code in controllers (HTTP), services (rules) and repositories (data access). Controllers contain no business rules.

Persistence

You are free to choose, but no database course or Docker knowledge is expected. **Recommended path, no installation needed:**

1. Create the project at start.spring.io with the dependencies *Spring Web*, *Spring Data JPA*, *H2 Database* and *Validation*.
2. Store the database in a file so data survives restarts: in `application.properties`, set `spring.datasource.url=jdbc:h2:file:./data/readingtracker` and `spring.jpa.hibernate.ddl-auto=update`. Add `data/` to your `.gitignore`.
3. Annotate your classes with `@Entity` and create one `JpaRepository` interface per entity. Spring generates the tables and SQL for you.
4. Load `seed.json` at startup with a `CommandLineRunner` that runs only when the database is empty. Put the file in `src/main/resources` and read it with Jackson's `ObjectMapper` (already included with Spring Web). On Spring Boot 4 this is Jackson 3: import `ObjectMapper` from `tools.jackson.databind`, not from `com.fasterxml.jackson.databind` as in Spring Boot 3 tutorials (annotations such as `@JsonIgnore` keep the

`com.fasterxml.jackson.annotation` package). Do not copy the seed IDs into your entities; let the database generate them. Insert in this order: `countries`, `themes`, `authors`, `books`, `readings`. While inserting, keep a Map from each seed ID to the saved entity, so that, for example, a book's `authorIds` can be turned into the saved `Author` objects. Reuse the same loading method for POST `/api/reset`.

a. **Note on IDs after a reset**. In Delivery 1, Reset data restores the seed exactly, so IDs start again from 1. In Delivery 2, the database never reuses IDs: after a reset, the same records come back with new, higher IDs (for example, the first author may become ID 102 instead of 1). This is expected and is not an error. Your code must therefore never depend on specific ID values: do not hard-code IDs in the frontend, and always use the IDs returned by the API. To start over with IDs from 1, stop the application, delete the `data/` folder, and start it again: the database is recreated empty and the seed is loaded.

Other options (a JSON file, SQLite, PostgreSQL, MongoDB) are accepted if the project still runs with the single documented command below. Docker is only required for FR-17.

Running the project

- The whole application must start with one command from the project folder: `./mvnw spring-boot:run` (`mvnw.cmd spring-boot:run` on Windows), or the Gradle equivalent, with no other installation besides Java 21 or newer. Commit the wrapper files (`mvnw`, `mvnw.cmd`, `.mvn/`).
- **Serve the frontend from Spring Boot** by placing it in `src/main/resources/static`. The application then opens at `http://localhost:8080` and you avoid cross-origin (CORS) errors. In `store.js`, use relative URLs such as `/api/books`, not `http://localhost:8080/api/books`.

8. Deliverables and submission

Each delivery must include a link to the GitHub repository and a self-assessment score (from 0.0 to 5.0 for each delivery). **Delivery 2 must also include a link to a video.** These must be submitted by email to **pagliares@bcc.unifal-mg.edu.br** by the due date (see Section 2 for the late submission policy).

The email subject must be **"Reading Tracker – Delivery 1 – [student names]"** (or **"Delivery 2"**, as applicable). For team submissions, one member must send the email and copy all other team members.

Note: The self-assessment score does not necessarily correspond to the grade assigned by the instructor.

Mark each delivery with a Git tag on the commit you want graded, and push the tag before the deadline:

```
git tag delivery-1

git push origin delivery-1
```

The instructor grades the tagged commit. If the tag is missing, the last commit pushed is graded; commits pushed after the deadline make the delivery late (Section 2).

Each delivery must contain:

- **Source code** organized as described in Sections 6 and 7, including `seed.json`.

- **README.md** with:
    - Team members (names and student IDs)
    - How to run the project, step by step, including the Java version you used
    - The requirement checklist: each Core FR (FR-01 to FR-07) marked as implemented, partially implemented or not implemented, plus any optional FR you attempted
    - **Delivery 2 only**: the list of API endpoints (method, path, purpose), with one request and response example per resource (authors, books, readings)
    - The AI usage statement (Section 10)
    - Delivery 2 only: the link to the delivery's video
- **Video (Delivery 2 only; maximum 15 minutes).** Submit one video demonstrating the completed application, uploaded to a platform where the instructor can watch it without signing in (e.g., YouTube unlisted or Google Drive with "anyone with the link" access).
    - First, demonstrate all implemented requirements in the running application, including that **data persists after an application restart**.
    - Then, walk through the main parts of the code, focusing on controllers, services, repositories, persistence configuration, and frontend API calls.
- **Videos longer than 15 minutes will be evaluated only up to minute 15.**

Commit throughout the work, not only at the end, with messages that describe the change. In teams, each member's contribution should be visible in the commit history.

9. Grading

Each delivery is worth 5.0 points, for a total of 10.0.

Delivery 1 – Frontend (5.0)

Criterion	What is assessed	Points
FR-01 Manage authors	Create, view, edit, delete; linked author cannot be deleted	0.4
FR-02 Manage books	Create, view, edit, delete; authors and themes selected from existing records	0.5
FR-03 Manage readings	Add, edit, delete readings; one-step wishlist entry; cascade delete with confirmation	0.5
FR-04 Status workflow	Status changes follow the Section 4 rules	0.4
FR-05 Reading log	Sortable list; missing values shown as "-"	0.4
FR-06 Validation	Every Section 4 rule enforced, with messages next to fields	0.5
FR-07 Seed data and reset	Seed loaded only on first use; data survives a reload; Reset data works	0.3

Code and interface quality	File and module organization, data access isolated in one module, semantic HTML, works on laptop and phone	1.0
`README`	Complete `README` with requirement checklist	1.0

A partially implemented requirement receives up to half of its points.

Delivery 2 – Full stack (5.0)

Criterion	What is assessed	Points
REST API	Minimum endpoints implemented, JSON, correct methods and status codes	1.5
Persistence	Data survives a restart; seed data loaded only on the first run; project starts with one command	1.0
Frontend integration	Every Delivery 1 feature works through the API, including Reset data; no `localStorage` for application data	1.0
Server-side validation and errors	Section 4 rules enforced on the server; 400 responses with field messages shown in the interface	0.5
Layered architecture	Controllers, services and repositories with clear responsibilities	0.5
`README` and video	Endpoint list, updated checklist; video (max. 15 min) shows the solution and explains the code	0.5

Optional exercises (not graded)

Students who would like to improve their web development skills are encouraged to implement FR-08 through FR-18 (see Section 13). These are optional exercises that are not graded. They do not change your grade, but you may list them in your `README` checklist and show them in your video.

Rules

- A project that does not start by following the `README` instructions (Delivery 2: the single command from Section 7) receives at most 50% of the points of that delivery.
- Using a forbidden framework or library (Section 3) removes the points of the criteria that depend on it.
- All team members receive the same grade, unless the commit history or the demo shows that a member did not contribute; in that case the instructor may grade that member individually.

10. AI usage and academic integrity

You may use AI tools (chatbots, code assistants, AI agents) in this assignment, provided you disclose it.

- **Disclose.** The `README` must include an AI usage section listing, for each tool you used, what you used it for and which parts of the project it affected, for example:

Tool	Used for	Parts affected
ChatGPT	Generated the first version of the CSS layout	`css/style.css`
GitHub Copilot	Autocomplete while writing validation functions	`js/validation.js`
Claude	Explained a JPA error	— (no code generated)

If you used none, say so.

- **Understand everything you submit.** After watching the video, the instructor may invite any team member to a short follow-up conversation and ask them to explain any part of the code. A member who cannot explain code they submitted **loses 40%** of the points of that delivery.
- **Undisclosed use** of AI tools, or copying code from other teams, is treated as academic dishonesty under the course rules.
- **Help between teams** is welcome for concepts and debugging, but each team writes its own code. Code adapted from tutorials, documentation or forums is allowed if you cite the source in a code comment.
- AI-generated code follows the same rules. Code produced by an AI tool must respect Section 3; a forbidden framework or library is not allowed because a tool suggested it.

11. Seed data

The instructor provides the reading log from the PDF as a single JSON file, `seed.json`. It contains 143 readings from 2021 to 2026 (115 Read, 12 Paused, 16 Want to read), with their books, authors, countries and themes. The data has already been cleaned: author names have one spelling, typos are corrected, and every controlled value uses the spelling from Section 4.

Where to put it. Delivery 1: in your frontend folder (for example, `data/seed.json`). Delivery 2: in `src/main/resources/seed.json`.

Structure

The file is an object with five arrays. Records refer to each other by ID.

Array	Fields
`countries`	`id`, `name`
`authors`	`id`, `name`, `countryId`, `birthDate`, `deathDate`
`themes`	`id`, `name`
`books`	`id`, `title`, `authorIds`, `publicationYear`, `form`, `mode`, `content`, `themeIds`
`readings`	`id`, `bookId`, `status`, `yearRead`, `grade`, `recommendedBy`, `priority`, `notes`

Example of one author, one book and one reading:

```
{
"id": 27, "name": "Fyodor Dostoevsky", "countryId": 22,
```

```
"birthDate": "1821-11-11", "deathDate": "1881-02-09"
}

{
"id": 10, "title": "Em Busca de Sentido", "authorIds": [96],
"publicationYear": 1946, "form": null, "mode": null,
"content": null, "themeIds": []
}

{
"id": 10, "bookId": 10, "status": "Read", "yearRead": 2021,
"grade": 7, "recommendedBy": null, "priority": null, "notes": null
}
```


Things to know about the data

- Missing values are `null`, and empty lists are `[]`. Many records are incomplete on purpose, because that is how the real log looks: 2 books have no authors, 12 have no publication year, most authors have no birth or death date, and books read before 2024 have no form, mode or theme.
- Dates are strings in the format `YYYY-MM-DD`. Publication years are integers, negative for works before Christ (for example, `-429`).
- Grades are numbers (`7`, `8.5`), never strings.
- IDs start at `1` in every array and are consecutive. Readings follow the original log within each year, but years go from oldest to newest (the PDF lists 2026 first).
- The country list is fixed (Section 4): your application shows these countries and does not create new ones.
- All data in the file satisfies the Section 4 rules, so your validation should accept every record.

Loading it

Delivery 1: see Section 6 (Data storage). Delivery 2: see Section 7 (Persistence, step 4).

Inspecting the database (Delivery 2, optional)

With the recommended H2 setup, you can browse your tables in the **H2 console**. On **Spring Boot 4** the console is a separate module: add the `spring-boot-h2console` dependency and set `spring.h2.console.enabled=true`. Then open `http://localhost:8080/h2-console`, set the JDBC URL to `jdbc:h2:file:./data/readingtracker`, user `sa`, `empty password`. Stop the application before deleting the `data/ folder`.

12. References

Suggested study materials covering the topics needed to complete the semester assignment. These topics will be addressed in Weeks 10–15 of the course, but you are encouraged to explore them earlier if you have already started working on your semester assignment.

Delivery 1 – Frontend

- Flexbox: Flexbox
- Responsive design: Responsive web design
- Tables: HTML table basics
- Forms and validation: Client-side form validation
- Functions: Functions · Test your skills: Functions
- Arrays: Arrays · Test your skills: Arrays
- JavaScript objects: Object basics
- JSON: Working with JSON
- Events: Events
- Client-side storage: Client-side storage
- Asynchronous JavaScript: Introducing asynchronous JavaScript · How to use promises
- Fetch API: Using the Fetch API
- JavaScript modules (optional): JavaScript modules
- Git tags: Git Basics – Tagging

Delivery 2 – Backend and frontend–backend integration

- Spring Boot: watch videos 36 through 44 in the course playlist on YouTube. The videos may use Spring Boot 3; the code works the same on Spring Boot 4, except for the H2 console (Section 11) and the Jackson imports (Section 7, Persistence, step 4).
- REST controllers: Building a RESTful Web Service
- Spring Data JPA: Accessing Data with JPA
- Many-to-many relationships: Many-To-Many Relationship in JPA
- H2 database: Spring Boot & H2 Tutorial

13. Optional exercises

Students who want to improve their web development skills may optionally implement the requirements below. They are not graded. FR-08 to FR-15 can be done in either delivery; FR-16 to FR-18 require the backend.

- **FR-08 Filters and search.** Filter the reading log by status, year read, author, form and theme; search by title or author name (partial, case-insensitive). Filters can be combined. In Delivery 2, filtering may be done in the frontend or through query parameters on `GET /api/readings`.
- **FR-09 Wishlist.** A dedicated view of readings with status `Want to read`, sorted by priority, showing who recommended each book.
- **FR-10 Books by century.** Books grouped by the century of original publication (e.g., 5th century BC, 17th century, 20th century), sorted by year inside each group.
- **FR-11 Authors by nationality.** Authors grouped by country, each showing birth and death dates, age (at death, or current age if alive) and the list of their books with at least one reading with status Read. Authors without a nationality appear in a separate group, Unknown.
- **FR-12 Statistics.** At least: number of books read per year; average grade per year; average grade by author nationality; average grade by form; top 10 highest-graded books.
- **FR-13 Charts.** At least two of the FR-12 statistics are shown as charts (`Chart.js` or similar).
- **FR-14 Autofill from Open Library.** While creating a book, search the Open Library API and fill in title, author and publication year from a result. Calling the API with `fetch` does not count as using a library.
- **FR-15 Import and export.** Export all data to CSV or JSON and import it back.

- **FR-16 Users and login.** Multiple users, each with a private list. You may use Spring Security for this exercise without prior approval.
- **FR-17 External database with Docker.** Run the persistence layer in a container using a `docker-compose.yml`.
- **FR-18 DTOs and global error handling.** The API never exposes JPA entities directly: requests and responses use DTO classes, with mapping done in a dedicated mapper class or method. All exceptions are handled in one place with `@RestControllerAdvice`, which returns the error format from Section 7 for `400`, `404` and `409` responses.

The views (FR-10, FR-11, FR-12 and FR-13) replace the sections the user currently maintains by hand (computed dynamically). They must be computed from the data, never typed.

Incomplete records. The seed data contains books with only a title, and many readings without a grade. Your views must handle these gaps instead of breaking or hiding them:

- A book without authors does not appear in FR-11 and is left out of averages by nationality.
- A book without a publication year appears in FR-10 under a separate group, Unknown century.
- Averages count only graded readings, and “books read per year” counts only readings with status Read.
- Wherever a value is missing, show it as “—” instead of `null`, undefined or NaN.

Good coding!

Rodrigo Martins Pagliares